

```

Thread 1.2 (src) In: syscall          L142  PC: 0x80001d64
    s9 = 0x180, s10 = 0xb0, s11 = 0x2}, ofile = {
--Type <RET> for more, q to quit, c to continue without paging--
    0x0 <repeats 16 times>}, cwd = 0x800187e0, name = {
    0x69, 0x6e, 0x69, 0x74, 0x0 <repeats 12 times>}}
(gdb) p /x p->trapframe->a7
$2 = 0xf
(gdb) p /x $sstatus
$3 = 0x200000022
(gdb) █

```

xv6 kernel is booting

```

hart 1 starting
hart 2 starting
scause=0xd sepc=0x80001d62 stval=0x0
panic: kerneltrap

```

```

Thread 1 hit Breakpoint 1, syscall ()
    at kernel/syscall.c:142
(gdb) p p->name
$1 = "init", '\000' <repeats 11 times>
(gdb) p p->pid
$2 = 1
(gdb) █

```

Q1) usertrap (in kernel/trap.c) is called syscall.

Q2) p->trapframe->a7 = 0xf. This is the system call number. 15 corresponds to SYS_sleep (check kernel/syscall.h). The value indexes into the syscalls[] table to dispatch to the correct handler.

Q3) sstatus = 0x200000022. The SPP bit (bit 8) is 0, so the previous mode was User mode.

Q4) Faulting instruction: lw a3, 0(zero) at 0x80001d62. The register corresponding to num is a3.

Q5) The kernel crashed because address 0 is not mapped in the kernel address space. scause = 0xd means Load Page Fault, and stval = 0x0 confirms the CPU tried to read from unmapped address 0.

Q6) Process name = "init", pid = 1

```
hart 1 starting
hart 2 starting
init: starting sh
$ sandbox 32768 - cat README
sandbox 32768 - cat README
cat: cannot open README
$ QEMU: Terminated
adeenz@Pooter:/mnt/c/Users/Adeenz/Downloads/xv6/xv6-labs-202
6$ ./grade-lab-syscall sandbox_mask
make: 'kernel/kernel' is up to date.
== Test sandbox_mask == sandbox_mask: OK (7.3s)
```

Applies for both moderate and easy sandboxing

STEPS:

- 1) Define sys_interpose in syscall.h
- 2) Define by int interpose(int mask, char *path) in user.h
- 3) Define by entry("interpose") in [usys.pl](#)
- 4) In proc structure, make an extra field of int syscall_mask to reject bitmasks in proc.h and char syscall_path[MAXPATH]
- 5) Add p->syscall_mask = 0 and p->syscall_path[0] = 0 in allocproc function in proc.c
- 6) Copy mask on fork in proc.c by adding np->syscall_mask = p->syscall_mask and strcpy(np->syscall_path, p->syscall_path)
- 7) Add sys_interpose in sysproc.c

```
uint64
sys_interpose(void)
{
    int mask;
    char path[MAXPATH];

    argint(0, &mask);
    argstr(1, path, MAXPATH);

    struct proc *p = myproc();
```

```

    p->syscall_mask = mask;
    safestrcpy(p->syscall_path, path, sizeof(p->syscall_path));

    return 0;
}

```

8) In syscall.c, declare by extern uint64 sys_interpose(void) and add corresponding array [SYS_interpose] sys_interpose

9) Modify syscall in syscall.c

```

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        if((p->syscall_mask & (1 << num)) &&
            num != SYS_open && num != SYS_exec) {
            p->trapframe->a0 = -1;
        } else {
            p->trapframe->a0 = syscalls[num]();
        }
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

```

```
hart 1 starting
hart 2 starting
init: starting sh
$ sandbox 32768 README grep xv6 README
sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
$ sandbox 32768 README grep xv6 x
sandbox 32768 README grep xv6 x
grep: cannot open x
$ make grade
make grade
exec make failed
$ QEMU: Terminated
```

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#define CHUNK (1024 * 1024)
#define MINLEN 4
#define MAXLEN 32

static int
is_secret_char(char c)
{
    return (c >= '0' && c <= '9') ||
           (c >= 'A' && c <= 'Z') ||
           (c >= 'a' && c <= 'z');
}

int
main(void)
{
    char *mem = sbrk(CHUNK);
```

```

if(mem == (char*)-1){
    printf("sbrk failed\n");
    exit(1);
}

int i = 0;
while(i < CHUNK){
    if(is_secret_char(mem[i])){
        int start = i;
        while(i < CHUNK && is_secret_char(mem[i]))
            i++;
        int len = i - start;
        int boundary_ok = (start == 0 || mem[start-1] == 0) && (i == CHUNK
|| mem[i] == 0);
        if(len >= MINLEN && len <= MAXLEN && boundary_ok){
            write(1, mem + start, len);
            write(1, "\n", 1);
        }
    } else {
        i++;
    }
}
exit(0);
}

```

```

adeenz@Pooter:/mnt/c/Users/Adeenz/Downloads/xv6/xv6-labs-202
6$ ./grade-lab-syscall attack
make: 'kernel/kernel' is up to date.
== Test attack == attack: OK (6.9s)

```

STEPS:

- 1) Define SYS_freemem in syscall.h
- 2) Make it accessible for users by adding int freemem in user.h
- 3) Generate assembly stub by entry("freemem") on [usys.pl](#)
- 4) Declare uint64 by freemem(void) in defs.h
- 5) Declare to validate user's arguments for kernel operation by
uint64 sys_freemem(void)
{return freemem();}

6) Declare in syscall.c

```
extern uint64 sys_freemem(void);
```

...

```
[SYS_freemem] sys_freemem,
```

7) Make a c file

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(void)
{
    printf("free memory: %d bytes\n", freemem());
    exit(0);
}
```

8) Join it in UPROG in makefile

9) Test it :)

```
xv6 kernel is booting
```

```
hart 2 starting
```

```
hart 1 starting
```

```
init: starting sh
```

```
$ freememtest
```

```
freememtest
```

```
free memory: 133238784 bytes
```

```
$
```